

Szczegółowy opis zatwierdzonych tematów pracy magisterskiej

Temat D2 – Dawid Olko • Temat P4 – Piotr Smoła

Rozszerzenie pracy inżynierskiej:
System rekomendacji produktów oparty na metodach hybrydowych

Niniejszy dokument stanowi rozwinięcie wcześniejszej listy propozycji tematów. Tematy to: **Pre-trenowane modele językowe (LLM) otwarto-źródłowe a klasyczne metody rekomendacji** (autor: Dawid Olko) oraz **Wnioskowanie przedziałowe (logika rozmyta typu drugiego, IT2-FLS) z biblioteką pyit2fls** (autor: Piotr Smoła).

W dokumencie przedstawiono: (1) analizę obecnego stanu aplikacji *SmartRecommender* – co już działa i co stanowi punkt wyjścia; (2) pełny, uszczegółowiony opis obu tematów – co, gdzie i w jaki sposób będzie realizowane; (3) uwzględnienie wskazówek promotora (język pracy i promptów, polskie modele jako punkt odniesienia, włączenie ANFIS, fundament wspólny obu prac); (4) przegląd rekomendowanych modeli LLM wraz z uzasadnieniem doboru; (5) analizę literatury dotyczącej minimalnej liczby użytkowników wymaganej dla wiarygodności statystycznej testów.

Spis treści

1	Ustalenia z promotorem	2
2	Stan obecny aplikacji <i>SmartRecommender</i>	2
3	Fundament wspólny obu prac	4
4	Temat D2 – pełny opis	5
	Pre-trenowane modele językowe (LLM) otwarto-źródłowe a klasyczne metody rekomendacji	5
5	Rekomendowane modele LLM dla tematu D2	13
6	Temat P4 – pełny opis	15
	Wnioskowanie przedziałowe (IT2-FLS) i hybryda neuronowo-rozmyta ANFIS a klasyczny system rozmyty	15
7	Wiarygodność statystyczna – minimalna liczba użytkowników	19
8	Podział prac i harmonogram ramowy	20

1 Ustalenia z promotorem

Poniżej ustalono dodatkowo cztery kwestie, które zostały uwzględnione w niniejszym opisie.

1. **Język pracy i promptów.** Praca magisterska (temat D2) zostanie napisana w języku polskim. Prompty kierowane do modeli językowych zostaną przygotowane w języku angielskim – jest to język, w którym modele uniwersalne osiągają najwyższą jakość, a jednocześnie utrzymuje to spójność z literaturą. Główną oś porównawczą stanowią modele anglojęzyczne (Llama, Mistral, Qwen i inne), natomiast polskie modele otwartoźródłowe (Bielik, PLLuM) zostają włączone jako ciekawy punkt odniesienia oraz przykład lokalnych inicjatyw badawczych.
2. **Polskie systemy i modele.** Zgodnie ze wskazówką promotora – „skoro istnieją polskie systemy, to należy je uwzględnić” – temat D2 obejmuje przegląd i empiryczną ocenę polskich modeli Bielik oraz PLLuM. Stanowi to wyraźny lokalny wkład pracy.
3. **Rozszerzenie tematu P4 o ANFIS.** Zgodnie z sugestią („można do badania włączyć ANFIS i porównać te trzy systemy”) zakres tematu P4 zostaje rozszerzony. Badanie obejmuje porównanie *trzech* systemów wnioskowania rozmytego: typu pierwszego (obecna implementacja Mamdaniego), typu drugiego przedziałowego (IT2-FLS, biblioteka `pyit2fls`) oraz hybrydy neuronowo-rozmytej ANFIS.
4. **Wiarygodność statystyczna i liczba użytkowników.** Liczba użytkowników wymagana dla istotności statystycznej zależy od minimalnych wymagań stosowanych testów – zwykle wystarcza kilkanaście do kilkudziesięciu użytkowników – oraz zalecił sprawdzenie liczb spotykanych w literaturze systemów rekomendacyjnych. Przegląd ten zawiera rozdział 7.

2 Stan obecny aplikacji *SmartRecommender*

Praca inżynierska zaowocowała w pełni działającym systemem e-commerce zintegrowanym w architekturze **Django + Django REST Framework + React + PostgreSQL**, z modułem rekomendacji obejmującym dziewięć metod. Praca magisterska nie buduje sklepu od nowa – rozszerza istniejący system o warstwę badawczo-eksperymentalną. Poniższy opis odzwierciedla faktyczny stan repozytorium i posłuży jako punkt odniesienia dla obu tematów.

Architektura i model danych

Backend (katalog `backend/home/`) opiera się na rozszerzonym modelu użytkownika z kontrolą dostępu opartą na rolach (admin / klient) oraz na zestawie modeli domenowych. Kluczowe encje to: `Product` (nazwa, cena, opis, kategorie, tagi – opis tekstowy jest wykorzystywany w filtracji opartej na zawartości), `Category`, `Order` i `OrderProduct` (oś czasowa zamówień), `Opinion` (oceny 1–5 oraz treść tekstowa, z ograniczeniem unikalności para

użytkownik–produkt), Specification (parametry techniczne) oraz Complaint (reklamacje).

Osobną grupę stanowią modele warstwy metod, których nazwy tabel rozpoczynają się od przedrostka `method_`: `UserInteraction` (zdarzenia: *view*, *click*, *add_to_cart*, *purchase*, *favorite*), `ProductSimilarity` (zapisane podobieństwa produkt–produkt, z rozróżnieniem typu: *collaborative* lub *content_based*), `ProductAssociation` (reguły asocjacyjne z miarami *support*, *confidence*, *lift*), `SentimentAnalysis` i `ProductSentimentSummary` (wynik analizy sentymentu opinii), `PurchaseProbability`, `UserPurchasePattern`, `SalesForecast`, `ProductDemandForecast`, `RiskAssessment` oraz `UserProductRecommendation`. Taki podział oznacza, że wyniki metod są *materializowane* w bazie – nie są przeliczane przy każdym żądaniu – co jest istotne dla powtarzalności eksperymentów.

Zaimplementowane metody rekomendacji

Lp.	Metoda i implementacja	Dane
1	Filtracja oparta na zawartości – TF-IDF, ważone cechy (kategorie, tagi, słowa kluczowe z opisu, kategoria cenowa), podobieństwo kosinusowe; klasa <code>CustomContentBasedFilter</code>	jawne, atrybuty
2	Filtracja kolaboratywna item-based – Adjusted Cosine Similarity (Sarwar i in. 2001), wyniki w <code>ProductSimilarity</code>	ukryte, transakcje
3	Wnioskowanie rozmyte typu Mamdaniego – klasa <code>SimpleFuzzyInference</code> , 6 reguł, zmienne: cena / jakość / popularność	mieszane
4	Wyszukiwanie rozmyte – klasa <code>CustomFuzzySearch</code> : odległość edycyjna, podobieństwo trigramowe i znakowe	tekst zapytań
5	Analiza sentymentu opinii – klasa <code>CustomSentimentAnalysis</code> : metoda leksykonowa z negacją, intensyfikаторami i analizą bigramów	tekst opinii
6	Łańcuchy Markowa – klasa <code>CustomMarkovChain</code> , przejścia między produktami / kategoriami	sekwencyjne
7	Klasyfikator naiwny Bayesa – klasa <code>CustomNaiveBayes</code> , predykcja prawdopodobieństwa zakupu	mieszane
8	Reguły asocjacyjne – klasa <code>CustomAssociationRules</code> : Apriori z reprezentacją bitmapową, parametry <code>min_support</code> , <code>min_confidence</code>	transakcje
9	Analiza sezonowości – komponent temporalny prognoz popytu i sprzedaży	czasowe

Wszystkie metody zaimplementowano autorsko w pliku `custom_recommendation_engine.py` (silnik rozmyty – w osobnym `fuzzy_logic_engine.py`), bez korzystania z gotowych bibliotek rekomendacyjnych. Każda metoda ma własny zestaw widoków API (np. `recommendation_views.py`, `association_views.py`, `sentiment_views.py`) oraz panel diagnostyczny w interfejsie administratora, co umożliwia niezależne uruchomienie i podgląd działania – jest to mocna podstawa pod eksperymenty porównawcze.

Zasilanie danymi – obecny seeder

Aplikacja zawiera komendę `python manage.py seed` (plik `seed.py`), zasilającą bazę danymi demonstracyjnymi sklepu komputerowego: kategorie, użytkownicy, produkty z realnymi opisami i specyfikacjami, opinie, zamówienia, reklamacje, a na końcu przeliczenie tabel pochodnych (sentyment, reguły asocjacyjne, podobieństwa, prognozy). Dane są jednak częściowo losowe i w niewielkiej liczbie – wystarczające do demonstracji systemu, lecz niewystarczające do rzetelnych eksperymentów. Stąd potrzeba fundamentu wspólnego opisanego w rozdziale 3.

3 Fundament wspólny obu prac

Poniżej przedstawiono zakres tego fundamentu w formie, którą obie prace współdziela. Jest to pierwszy etap realizowany wspólnie przez obu autorów; dopiero na nim budowane są osobne rozdziały eksperymentalne.

1. **Badawczy seeder danych** – nowa komenda `python manage.py seed_research`, działająca obok istniejącego `seed` (bez kasowania go, bez zmian schematu bazy). Seeder generuje parametryzowaną liczbę użytkowników z *personami zakupowymi* (np. gracz, biuro, twórca treści, kupujący prezent, entuzjasta), spójne tematycznie koszyki, oś czasową zamówień oraz strumienie danych jawnych (opinie) i ukrytych (interakcje). Stały *seed* liczbowy gwarantuje powtarzalność wyników. Parametry siły sygnału i poziomu szumu są jawne i kontrolowane.
2. **Polecenie** `prepare_research_db` – po zasileniu bazy zawsze wykonuje to samo przeliczenie tabel pochodnych (macierz podobieństw, reguły asocjacyjne, agregaty sentymentu), tak aby każdy eksperyment startował z identycznego stanu bazy.
3. **Wspólny moduł metryk** (np. `home/experiments/metrics.py`) – ten sam podział temporalny (ostatni ustalony procent osi czasu = zbiór testowy) oraz te same metryki rankingowe: Precision@K , Recall@K , nDCG@K , dla raz ustalonego K .
4. **Miejsce zapisu wyników** – model `ExperimentRun` (parametry konfiguracji, metryki, czas wykonania, znacznik czasu) lub eksport do plików CSV – aby wyniki nie ginęły w logach konsoli i były gotowe do analizy statystycznej.

Dzięki wspólnemu fundamentowi temat D2 i temat P4 korzystają z tego samego zbioru danych, tego samego podziału i tych samych metryk – co umożliwia bezpośrednie porównanie wyników w sekcji wspólnej obu prac.

4 Temat D2 – pełny opis

Temat D2

Źródło: autorski (Dawid Olko) • Temat zatwierdzony

Pre-trenowane modele językowe (LLM) otwarto-źródłowe a klasyczne metody rekomendacji

Cel i pytanie badawcze

Temat odpowiada na aktualne pytanie literatury rekomendacyjnej: czy nowoczesne, otwarto-źródłowe duże modele językowe (LLM) mogą realnie konkurować z klasycznymi metodami rekomendacji, a jeśli tak – w jakim trybie wykorzystania, jakim kosztem i z jakim ryzykiem błędów. Praca porównuje rekomendacje generowane przez dostępne klasyczne metody z pracy inżynierskiej z rekomendacjami uzyskanymi z modeli LLM, w jednolitym protokole ewaluacji.

Trzy paradygmaty wykorzystania LLM

Praca eksploruje trzy sposoby włączenia modelu językowego do systemu rekomendacji – od najprostszego do najbardziej złożonego:

1. **Bezpośrednia rekomendacja / re-ranking (prompting).** Model otrzymuje w prompcie historię zakupów użytkownika oraz *listę produktów-kandydatów* – zawężoną wcześniej przez tanią metodę klasyczną (filtracja oparta na zawartości lub kolaboratywna), aby model nie musiał przeszukiwać całego katalogu. Zadaniem modelu jest uporządkowanie listy. Badane są warianty zero-shot i few-shot. Zgodnie z uwagą, prompty są przygotowane w języku angielskim.
2. **Reprezentacje wektorowe opisów (embeddings).** Model językowy (lub dedykowany model embeddujący) oblicza wektorowe reprezentacje opisów produktów i treści opinii. Reprezentacje są indeksowane w lekkiej bazie wektorowej, a rekomendacja sprowadza się do wyszukiwania najbliższych sąsiadów. Ten paradygmat naturalnie radzi sobie z problemem zimnego startu produktu – nowy produkt z opisem jest rekomendowalny natychmiast, mimo braku historii zakupów.
3. **Retrieval-Augmented Generation (RAG).** Kontekst przekazywany do modelu generującego ranking jest wzbogacany o powiązane opinie i specyfikacje techniczne, pobierane z bazy wektorowej. Hipoteza: RAG ogranicza halucynacje względem prostego promptingu, ale podnosi opóźnienie odpowiedzi.

Spójność tematów D2 i P4

Oba tematy badają tę samą rzecz – czy nowocześniejsze, bardziej złożone podejście realnie poprawia jakość rekomendacji, czy jedynie podnosi koszt. D2 zadaje to pytanie

dla modeli językowych, P4 – dla formalnego modelowania niepewności (logika rozmyta typu drugiego, ANFIS). Oba korzystają z tego samego seeder badawczego, tego samego podziału temporalnego i tych samych metryk, co pozwala zestawić wyniki w jednym wspólnym rozdziale.

Zakres prac implementacyjnych

Powstaje moduł rekomendacji oparty na LLM, działający równolegle do istniejących modułów – bez ingerencji w działający system. Moduł składa się z trzech klas odpowiadających trzem paradygmatom:

- **Klasa promptująca** – buduje prompt zawierający historię zakupów i listę kandydatów, wywołuje model, parsuje odpowiedź. Odpowiedź modelu jest walidowana strukturalnie (np. Pydantic v2); w razie niepoprawnej odpowiedzi system stosuje fallback do rankingu bazowego.
- **Klasa embeddująca** – liczy reprezentacje wektorowe opisów i opinii, indeksuje je, obsługuje zapytania semantyczne (wyszukiwanie kNN).
- **Klasa RAG** – pipeline pobierający kontekst i wzbogacający prompt przed wywołaniem modelu generującego ranking.

Wszystkie klasy korzystają wyłącznie z modeli otwarto-źródłowych, co zapewnia pełną reprodukowalność i niezależność od polityk komercyjnych dostawców.

Stack technologiczny

Modele LLM uniwersalne	Llama 4 / 4 Scout, Mistral (Medium 3.5 / Large 3, Apache 2.0), Qwen 3.5, GLM-5.1, DeepSeek V4 – patrz rozdział 5
Modele LLM lokalne	Mistral Small (24B), Gemma 4, Phi-4 (14B), Qwen 3.5 w mniejszych wariantach – działają na jednej karcie GPU
Modele LLM polskojęzyczne	Bielik v3 (7B i 11B, SpeakLeash, Apache 2.0), PLLuM (8B–70B, konsorcjum OPI PIB) – jako punkt odniesienia
Reprezentacje wektorowe	sentence-transformers; modele wielojęzyczne BGE / E5 (porównanie modelu wielojęzycznego z anglojęzycznym)
Baza wektorowa	pgvector (rozszerzenie PostgreSQL – bez nowych usług w stacku) lub Qdrant / ChromaDB w trybie lokalnym
Framework RAG	LangChain lub LlamaIndex
Inferencja	Ollama, llama.cpp, vLLM lub Hugging Face Transformers
Backend	Django + DRF – nowy moduł rekomendacji LLM równolegle do istniejących

Innowacyjność

Wkład pracy obejmuje cztery elementy. Po pierwsze, systematyczne porównanie modeli LLM różnej skali i pochodzenia z klasycznymi metodami rekomendacji na tym samym zbiorze danych e-commerce. Po drugie, analiza kompromisu między jakością rekomendacji a kosztem obliczeniowym (opóźnienie odpowiedzi, zapotrzebowanie na pamięć GPU). Po trzecie, ocena współczynnika halucynacji – częstości, z jaką model rekomenduje produkty nieistniejące w katalogu lub przypisuje im fałszywe atrybuty; jest to metryka rzadko raportowana, a krytyczna dla wdrożeń produkcyjnych. Po czwarte, badanie wpływu języka oraz ocena polskich modeli Bielik i PLLuM jako lokalnie dostępnej, taniej alternatywy – co stanowi wyraźny lokalny wkład pracy.

Hipotezy badawcze

- **H1.** Model LLM operujący na liście kandydatów dostarczonej przez tanią metodę zawężającą osiąga wyższą metrykę $nDCG@K$ niż ta metoda samodzielnie.
- **H2.** Jakość rekomendacji rośnie wraz ze skalą modelu, ale punkt nasycenia leży poniżej największych dostępnych modeli – stosowanie modeli największych nie opłaca się, jeśli mniejsze dają porównywalny wynik.
- **H3.** Polski model Bielik v3 11B operujący na polskich opisach produktów osiąga jakość niegorszą niż wielokrotnie większe modele uniwersalne, przy znacząco niższym koszcie obliczeniowym.
- **H4.** Pipeline RAG zmniejsza współczynnik halucynacji względem prostego promptingu, lecz podnosi opóźnienie odpowiedzi.

Plan eksperymentu

Przygotowanie reprezentatywnego zbioru zapytań rekomendacyjnych z badawczego zbioru danych. Generacja rekomendacji przez wybrane modele otwarto-źródłowe (od czterech do sześciu modeli reprezentujących różne klasy wielkości i pochodzenia, w tym co najmniej jeden polski). Generacja rekomendacji przez wszystkie metody klasyczne oraz wybrane warianty hybrydowe (LLM nad metodą klasyczną). Pomiar metryk $Precision@K$, $nDCG@K$, współczynnika halucynacji, średniego opóźnienia odpowiedzi i kosztu zasobowego. Ocena ekspercka na próbie kontrolnej w celu walidacji jakościowej. Analiza statystyczna – test Friedmana dla porównania wielu metod, z testem post-hoc (np. Nemenyiego) dla par metod.

Koszty modeli i nakład pracy na wdrożenie

Świadomie wybieram wyłącznie modele otwarto-źródłowe, więc **za same modele nie płacę nic** – pobieram je bezpłatnie z platformy Hugging Face i uruchamiam lokalnie na własnym sprzęcie. W odróżnieniu od komercyjnych API (OpenAI, Anthropic, Google), gdzie płaci się za każde tysiąc tokenów, tutaj nie ma żadnych opłat za zapytanie ani limitów – mogę wykonać dowolnie wiele przebiegów eksperymentu, co jest istotne dla powtarzalności

badania. Jedyny realny koszt to prąd i czas pracy mojego komputera.

Pozostają trzy potencjalne, opcjonalne źródła kosztów – żadne nie jest konieczne do realizacji pracy:

- **Modele zbyt duże na moją kartę graficzną** (rzędu 70B parametrów i większe). Jeśli zdecyduję się je włączyć do porównania, mogę skorzystać z usług hostingu inferencji rozliczanych za użycie (np. Together AI, Groq, OpenRouter) – koszt jednego pełnego przebiegu eksperymentu na takim modelu to zwykle kilka–kilkanaście złotych, a całość mieści się w kilkudziesięciu złotych. Alternatywnie – bezpłatnie – uczelniany klaster obliczeniowy lub darmowy limit GPU w środowisku Google Colab.
- **Wynajem GPU w chmurze** (np. RunPod, Vast.ai) – kilka złotych za godzinę, gdyby mój sprzęt okazał się niewystarczający. Przy planowanym zestawie 4–6 modeli mieszczących się lokalnie nie przewiduję takiej potrzeby.
- **Modele polskie** – Bielik (licencja Apache 2.0) i PLLuM są w pełni darmowe i uruchamiają się lokalnie.

Czas nauki. Nie jest wymagane trenowanie ani dostrajanie modeli (fine-tuning) – modele wykorzystuję w trybie gotowej inferencji (zero-shot i few-shot), co eliminuje najtrudniejszą i najbardziej czasochłonną część pracy z modelami językowymi. Realny nakład na opanowanie potrzebnych narzędzi szacuję następująco: uruchomienie modelu lokalnie przez Ollamę – kwestia kilku godzin (instalacja, pobranie modelu, pierwsze zapytanie); konstrukcja promptów i walidacja odpowiedzi – kilka dni iteracyjnej pracy; podstawy RAG i bazy wektorowej (pgvector, biblioteka LangChain lub LlamaIndex) – około tygodnia. Najwięcej czasu pochłonie nie nauka narzędzi, lecz właściwe eksperymenty: dobór promptów, przebiegi pomiarowe i analiza wyników. Całość mieści się w typowym harmonogramie pracy magisterskiej i nie wymaga wcześniejszego doświadczenia z modelami językowymi.

W skrócie – koszty

Modele: **0 zł** (otwarto-źródłowe, uruchamiane lokalnie). Brak opłat za zapytania, brak limitów. Opcjonalny koszt tylko gdybym chciał włączyć modele największej skali – rzędu kilkudziesięciu złotych za hosting rozliczany za użycie, albo bezpłatnie przez klaster uczelni lub Google Colab. Trening modeli nie jest potrzebny.

Metodyka badania – jak dane są porównywane i jak powstają wnioski

Praca ma charakter badawczy – jej trzon stanowi systematyczne, mierzalne porównanie metod i modeli, zakończone wnioskami popartymi analizą statystyczną. Poniżej opisano, jak to porównanie jest przeprowadzane krok po kroku.

Krok 1 – punkt odniesienia (ground truth). Aby w ogóle dało się ocenić, czy rekomendacja jest trafna, potrzebny jest obiektywny wzorzec „prawidłowej odpowiedzi”. Stosuję *podział temporalny*: oś czasu zakupów każdego użytkownika dzielona jest na część uczącą (wcześniejsze ok. 80% historii) i część testową (ostatnie ok. 20%). Metoda lub model „widzi”

wyłącznie część uczącą i na jej podstawie generuje ranking; ranking ten jest następnie konfrontowany z produktami, które użytkownik *faktycznie kupił* w odciętej części testowej. Produkt obecny w przyszłych zakupach to trafienie. Podział jest deterministyczny (stały *seed*), więc każde porównanie jest powtarzalne i nie wymaga udziału ludzi.

Krok 2 – wspólny protokół. Wszystkie metody klasyczne i wszystkie modele LLM oceniane są w *identycznych* warunkach: ten sam badawczy zbiór danych, ten sam podział temporalny, ta sama wartość K , te same metryki. To warunek rzetelności – bez niego porównanie nie miałoby wartości naukowej. Protokół realizuje wspólny moduł metryk (`home/experiments/metrics.py`) z fundamentu wspólnego.

Krok 3 – metryki. Dla każdego użytkownika i każdej metody (lub modelu) obliczane są:

- **Precision@K** – jaka część z K poleconych produktów okazała się trafiona;
- **Recall@K** – jaką część rzeczywistych przyszłych zakupów udało się odzyskać;
- **nDCG@K** – czy trafione produkty znalazły się wysoko w rankingu (pozycja ma znaczenie – trafienie na miejscu 1. liczy się więcej niż na 10.);
- **współczynnik halucynacji** – odsetek rekomendacji wskazujących produkty spoza katalogu lub z fałszywymi atrybutami (metryka specyficzna dla LLM);
- **opóźnienie odpowiedzi i koszt zasobowy** – średni czas wygenerowania rekomendacji oraz zapotrzebowanie na pamięć GPU.

Krok 4 – trzy osie porównania. Badanie zestawia wyniki na trzech płaszczyznach:

- *LLM kontra metody klasyczne* – np. Bielik kontra filtracja kolaboratywna kontra reguły asocjacyjne, na tych samych użytkownikach (główne pytanie pracy);
- *LLM kontra LLM* – porównanie modeli między sobą: Qwen, Mistral, Bielik, Phi-4 i inne (weryfikacja hipotez H2 i H3 o wpływie skali i pochodzenia modelu);
- *paradygmat kontra paradygmat* – prompting kontra embeddings kontra RAG (weryfikacja hipotezy H4).

Krok 5 – analiza statystyczna. Sama tabela średnich nie wystarcza – różnica może być przypadkowa. Każda metoda daje wektor wyników (po jednej wartości metryki na każdego użytkownika testowego). **Test Friedmana** sprawdza, czy różnice między wszystkimi porównywanymi metodami są istotne statystycznie (przy poziomie istotności $\alpha = 0,05$). Jeśli test wykaże istotność, **test post-hoc Nemenyiego** wskazuje, które konkretnie pary metod różnią się istotnie. Pozwala to przejść od stwierdzenia „model A wygląda lepiej” do „model A jest istotnie lepszy od modelu B ($p = 0,03$)”. Dla porównań par metod stosowany jest dodatkowo test Wilcoxona dla par powiązanych.

Krok 6 – wnioski. Wyniki podsumowuje ranking metod według jakości oraz wykres kompromisu *jakość kontra koszt* (front Pareto: nDCG@K na jednej osi, opóźnienie odpowiedzi na drugiej). Wniosek pracy ma postać konkretnej, popartej liczbami rekomendacji – na przykład: „dla sklepu tej skali model Bielik 11B w trybie promptingu osiąga około 90% jakości największego modelu uniwersalnego przy ułamku kosztu obliczeniowego, a pipeline

RAG nie poprawia wyniku w sposób istotny statystycznie”. Tak sformułowany wniosek odpowiada wprost na pytanie badawcze pracy i jest weryfikowalny.

Aspekt badawczy w skrócie

Praca nie jest wyłącznie wdrożeniem modeli LLM – jest **badaniem porównawczym**. Mierzy te same metryki dla wszystkich metod w identycznych warunkach, sprawdza istotność różnic testami statystycznymi (Friedman, post-hoc Nemenyi, Wilcoxon) i kończy się wnioskami popartymi liczbami: który model i który paradygmat daje najlepszy stosunek jakości do kosztu. To właśnie stanowi naukową wartość tematu D2.

Przeprowadzenie testów i panel administratora

Aby eksperymenty były wygodne do uruchamiania i przejrzyste w odbiorze, warstwa badawcza tematu D2 zostanie zintegrowana z istniejącym panelem administratora – analogicznie do obecnych paneli diagnostycznych poszczególnych metod. Powstanie nowa zakładka „Eksperymenty LLM”, a w niej:

- **Uruchamianie eksperymentu z panelu** – formularz pozwalający wybrać badane modele, paradygmat (prompting / embeddings / RAG), wartość K oraz podzbiór użytkowników testowych, a następnie uruchomić przebieg jednym przyciskiem. Eksperyment wykonuje się w tle (zadanie asynchroniczne), a panel pokazuje pasek postępu – nie trzeba uruchamiać niczego z terminala.
- **Tabela wyników** – każdy zakończony przebieg trafia jako wiersz do modelu `ExperimentRun` i jest wyświetlany w panelu: nazwa modelu, paradygmat, metryki Precision@K , nDCG@K , współczynnik halucynacji, średnie opóźnienie odpowiedzi, znacznik czasu. Wyniki można sortować i filtrować.
- **Wykresy porównawcze** – panel rysuje zestawienie metryk dla wszystkich przebiegów (np. wykres słupkowy nDCG@K dla kolejnych modeli oraz wykres jakości kontra opóźnienie), co daje natychmiastowy obraz, który model wypada najlepiej.
- **Eksport do CSV** – przycisk „Eksportuj wyniki” pobiera wszystkie przebiegi (lub wybrane) w postaci pliku CSV gotowego do dalszej analizy statystycznej w środowisku zewnętrznym (Python, R, arkusz kalkulacyjny). Eksport CSV jest też dostępny bezpośrednio przez endpoint API, na wypadek pracy wsadowej.
- **Podgląd pojedynczej rekomendacji** – dla wybranego użytkownika panel pokazuje obok siebie listę kandydatów, ranking zwrócony przez model oraz – przy paradygmacie RAG – fragmenty kontekstu (opinie, specyfikacje), które trafiły do promptu. Ułatwia to diagnostykę i jest atrakcyjnym materiałem na obronę.

Takie podejście oznacza, że całe badanie da się przeprowadzić i przeglądać „klikając” w panelu, a wyniki w każdej chwili wyeksportować do CSV – bez grzebania w konsoli.

Widoczność zmian po stronie frontendu

Tak – zmiany będą widoczne w interfejsie, i to w dwóch miejscach. Po pierwsze, w **panelu administratora** pojawi się opisana wyżej zakładka eksperymentów. Po drugie, na **stronie sklepu** rekomendacje generowane przez modele LLM zostaną podpisane jako kolejna, równoprawna opcja w istniejącym mechanizmie wyboru algorytmu rekomendacji.

Obecnie aplikacja pozwala wskazać aktywny algorytm rekomendacji (model `RecommendationSettings` z wariantami: filtracja kolaboratywna, oparta na zawartości, logika rozmyta), a sekcje rekomendowanych produktów na stronach renderują się zgodnie z tym wyborem. W ramach tematu D2 lista dostępnych algorytmów zostanie rozszerzona o opcję opartą na LLM – po jej wybraniu sekcje rekomendacji (np. „Polecane dla Ciebie”, „Produkty dopasowane”) będą wyświetlać produkty uszeregowane przez model językowy. Dzięki temu admin może w panelu przełączyć sklep na rekomendacje LLM dokładnie tak samo, jak dziś przełącza między metodami klasycznymi, a efekt jest natychmiast widoczny dla klienta na stronie. Pozwala to też pokazać działanie na żywo podczas obrony – przełączenie algorytmu i porównanie, jak zmienia się zawartość sekcji rekomendacji.

Sposób podpięcia modeli i miejsce konfiguracji promptów

Modele językowe podpinam do aplikacji przez *warstwę inferencji* – lokalny serwer (Ollama lub vLLM) udostępniający model pod adresem HTTP, z którym backend Django komunikuje się tak samo, jak z dowolnym innym serwisem. Adres serwera i nazwa modelu trafiają do konfiguracji aplikacji (plik ustawień / zmienne środowiskowe), więc zmiana modelu nie wymaga modyfikacji kodu – wystarczy wskazać inny model w konfiguracji lub, docelowo, w panelu administratora.

Prompty **nie są zaszyte na sztywno w kodzie**. Tekst promptu rozkłada się na dwie części. Część stała – *szablon promptu* (instrukcja zadania, format oczekiwanej odpowiedzi, ewentualne przykłady few-shot) – przechowywana jest jako wersjonowany szablon: w pierwszej wersji w pliku konfiguracyjnym repozytorium, docelowo w bazie danych, co umożliwi edycję szablonu wprost z panelu administratora bez ponownego wdrażania aplikacji. Część zmienna – historia zakupów użytkownika i lista produktów-kandydatów – jest wstawiana do szablonu dynamicznie przy każdym żądaniu. Takie rozdzielenie ma trzy zalety: pozwala porównywać różne wersje promptów w eksperymencie (a wynik każdego przebiegu w tabeli `ExperimentRun` zapisuje, której wersji promptu użyto – co gwarantuje powtarzalność), umożliwia łatwą edycję promptów bez dotykania kodu oraz utrzymuje zgodność z ustaleniem językowym – szablon redaguję po angielsku, mimo że dane produktów pozostają w języku polskim.

Literatura

- Wu, L. i in. (2024). A Survey on Large Language Models for Recommendation. *arXiv:2305.19860*.
- Hou, Y. i in. (2024). Large Language Models are Zero-Shot Rankers for Recommender Systems. *ECIR '24*.

- Bao, K. i in. (2023). TALLRec: An Effective and Efficient Tuning Framework to Align LLM with Recommendation. *RecSys '23*.
- Lewis, P. i in. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*.
- Reimers, N., Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP*.
- Ociepa, K. i in. (2025). Advancing Polish Language Modeling through Tokenizer Optimization in the Bielik v3 Series. *arXiv*.

5 Rekomendowane modele LLM dla tematu D2

Dobór modeli do badania kieruje się czterema kryteriami: (1) **licencja otwarto-źródłowa** pozwalająca na użycie badawcze i reprodukcję; (2) **możliwość uruchomienia lokalnego** – przynajmniej części modeli – bez zależności od komercyjnego API; (3) **reprezentatywność skali** – modele małe, średnie i duże, aby zbadać hipotezę o punkcie nasycenia; (4) **aktualność** – modele stanowiące stan wiedzy w pierwszej połowie 2026 roku. Poniżej zestaw rekomendowany, podzielony na trzy osie porównawcze.

Oś główna – modele uniwersalne anglojęzyczne

Model	Skala / licencja	Uzasadnienie doboru
Qwen 3.5	wiele wariantów; Apache 2.0	Najlepszy stosunek jakości do kosztu i otwartości w 2026 r.; dostępny w wariantach od kilku do kilkuset miliardów parametrów – pozwala badać hipotezę H2 (punkt nasycenia) w obrębie jednej rodziny modeli.
Llama 4 / 4 Scout	MoE; licencja Meta	Najszerzej wdrażana rodzina modeli otwartych; wariant Scout dobrze radzi sobie z długim kontekstem – istotne dla RAG i długich list kandydatów.
Mistral (Medium 3.5 / Large 3)	41B aktywnych (Large 3); Apache 2.0	Model europejski, w pełni otwarta licencja Apache 2.0; dobry punkt odniesienia dla modeli „frontier” przy zachowaniu reprodukowalności.
DeepSeek V4	MoE	Silny model do długiego kontekstu; reprezentuje klasę dużych modeli MoE dostępnych do uruchomienia lub przez API uczelni.

Oś lokalna – modele uruchamialne na jednej karcie GPU

Modele tej osi służą do zbadania tezy, że rekomendacja LLM jest wykonalna bez infrastruktury chmurowej – co ma bezpośrednie znaczenie praktyczne dla mniejszego sklepu e-commerce.

Model	Skala	Uzasadnienie doboru
Mistral Small	24B	Mieści się w 16 GB VRAM po kwantyzacji 4-bit; dobry kompromis jakość/rozmiar dla wariantu lokalnego.
Gemma 4	ok. 27B	Rodzina Google o silnych wynikach na zadaniach językowych; alternatywa dla Mistral Small w porównaniu modeli lokalnych.
Phi-4	14B	Mały, „gęsty” model o dużej efektywności – reprezentuje dolną granicę skali; mieści się w VRAM po kwantyzacji 8-bit.

Oś polska – punkt odniesienia i lokalny wkład pracy

Bielik to seria otwarto-źródłowych modeli językowych dla języka polskiego, rozwijana przez społeczność SpeakLeash przy wsparciu Akademii Górniczo-Hutniczej i innych uczelni. Modele Bielik v3 (7B i 11B) powstały przez kontynuację treningu (*continual pre-training*) na bazie modelu Llama, z użyciem korpusu polskich tekstów liczącego dziesiątki miliardów tokenów. Innowacją jest autorski tokenizer zoptymalizowany pod język polski, redukujący liczbę tokenów potrzebnych do reprezentacji polskiego tekstu – co przekłada się na krótszy czas inferencji. Modele są dostępne na licencji Apache 2.0.

PLLuM (Polish Large Language Model) to inicjatywa finansowana ze środków publicznych, realizowana przez konsorcjum obejmujące OPI PIB, Politechnikę Gdańską i inne jednostki. PLLuM oferuje modele w rozmiarach od 8B do 70B parametrów, trenowane lub dostrajane na polskojęzycznych danych ogólnych i specjalistycznych.

W temacie D2 – zgodnie z ustaleniem językowym – modele polskie pełnią rolę *punktu odniesienia*: pozwalają sprawdzić, czy model trenowany na polskich danych daje przewagę przy polskojęzycznych opisach produktów, mimo że prompty operacyjne sformułowane są po angielsku. Hipoteza H3 zakłada, że Bielik v3 11B dorównuje wielokrotnie większym modelom uniwersalnym przy ułamku kosztu – jest to teza interesująca naukowo i o wyraźnym lokalnym charakterze.

Uwaga praktyczna – dobór końcowy

Do badania nie trzeba używać wszystkich wymienionych modeli. Rekomendowany zestaw roboczy to **4–6 modeli**: dwa z osi głównej (np. Qwen 3.5 i Mistral), jeden–dwa lokalne (np. Mistral Small lub Phi-4) oraz co najmniej jeden polski (Bielik v3 11B). Taki dobór pokrywa pełne spektrum skali i pochodzenia, a jednocześnie utrzymuje liczbę warunków eksperymentu na poziomie wykonalnym – co jest istotne dla mocy statystycznej testów (patrz rozdział 7).

6 Temat P4 – pełny opis

Temat P4

Źródło: promotor (Piotr Smoła) • Temat zatwierdzony

Wnioskowanie przedziałowe (IT2-FLS) i hybryda neuronowo-rozmyta ANFIS a klasyczny system rozmyty

Cel i pytanie badawcze

Temat stanowi rozszerzenie istniejącego w aplikacji modułu wnioskowania rozmytego typu pierwszego (klasyczny system Mamdaniego, klasa `SimpleFuzzyInference`). Pytanie badawcze: czy formalne, bardziej zaawansowane modelowanie niepewności realnie poprawia jakość rekomendacji w warunkach rzadkich danych, czy jedynie zwiększa koszt obliczeniowy bez wymiernych korzyści. Zgodnie z sugestią promotora badanie obejmuje *trzy* systemy: typu pierwszego, typu drugiego przedziałowy (IT2-FLS) oraz hybrydę neuronowo-rozmytą ANFIS.

Punkt wyjścia – istniejący system rozmyty

W obecnej implementacji (`fuzzy_logic_engine.py`) system typu pierwszego operuje na trzech zmiennych językowych: *cena* (zbiory: tani / średni / drogi – funkcje trójkątne i trapezowe), *jakość* (niska / średnia / wysoka, na podstawie oceny 1–5) oraz *popularność* (niska / średnia / wysoka, na podstawie liczby wyświetleń). Baza reguł liczy sześć reguł (np. „R1: wysoka jakość w dobrej cenie”), agregacja przesłanek odbywa się normą t (minimum), a defuzyfikacja – przez ważoną średnią aktywacji reguł. Ten system jest bezpośrednim punktem odniesienia dla nowych wariantów – nie wymaga przebudowy, służy jako baseline.

Trzy badane systemy

1. **System typu pierwszego (T1-FLS)** – istniejący system Mamdaniego. Stopień przynależności jest liczbą – np. cena pewnego produktu należy do zbioru „tani” w stopniu dokładnie 0,5.
2. **System typu drugiego przedziałowy (IT2-FLS)** – modeluje niepewność samej funkcji przynależności. Zamiast jednej liczby stopień przynależności jest *przedziałem* – np. od 0,4 do 0,6. Każdy zbiór rozmyty typu drugiego charakteryzowany jest parą funkcji wyznaczających obszar niepewności (*Footprint of Uncertainty*, FOU). Lepiej oddaje to rzeczywistą niepewność danych e-commerce: ceny zmieniają się dynamicznie, oceny są subiektywne, opinii bywa mało. Implementacja korzysta z biblioteki `pyit2fls`, dostarczającej klasy zbiorów IT2FS, systemy Mamdani i TSK oraz algorytmy redukcji typu i defuzyfikacji Karnika-Mendela.
3. **Hybryda neuronowo-rozmyta ANFIS** – system, w którym parametry funkcji przynależności i wagi reguł nie są ustalane ręcznie przez eksperta, lecz *uczone* z danych

(feedbacku użytkowników) algorytmem uczenia hybrydowego. ANFIS eliminuje potrzebę ręcznego strojenia systemu eksperckiego. Włączenie ANFIS do badania jest bezpośrednią realizacją sugestii promotora – pozwala porównać trzy podejścia: ekspercką wiedzę stałą (T1), ekspercką wiedzę z modelowaniem niepewności (IT2) oraz wiedzę uczoną z danych (ANFIS).

Zakres prac implementacyjnych

Powstają dwa nowe moduły wnioskowania, działające równolegle do istniejącego modułu typu pierwszego:

- **Moduł IT2-FLS** – definicja zbiorów rozmytych typu drugiego dla wszystkich pojęć językowych systemu (cena, jakość, popularność), każdy z wyznaczonym obszarem niepewności FOU; przepisanie istniejącej bazy sześciu reguł do postaci typu drugiego (klasa `IT2Mamdani` z `pyit2fls`); defuzyfikacja algorytmem Karnika-Mendela.
- **Moduł ANFIS** – struktura sieci adaptacyjnej odwzorowująca system rozmyty, z parametrami funkcji przynależności uczonymi z danych; trening na feedbacku użytkowników z badawczego zbioru danych.
- **Panel diagnostyczny** – rozszerzenie istniejącego panelu logiki rozmytej w interfejsie administratora o wizualizację obszaru niepewności FOU, aktywacji reguł górnych i dolnych oraz przedziału wyjściowego.
- **Mechanizm porównania równoległego** – uruchomienie wszystkich trzech systemów na tych samych danych wejściowych i zapis wyników do wspólnej tabeli eksperymentów.

Stack technologiczny

System typu drugiego	biblioteka <code>pyit2fls</code> (<code>pip install pyit2fls</code>)
ANFIS	implementacja na bazie PyTorch / TensorFlow lub dedykowanej biblioteki ANFIS
Numeryka	NumPy, SciPy
Wizualizacja FOU	matplotlib – wykresy obszaru niepewności
Backend	Django – nowe moduły logiki rozmytej obok istniejącego <code>fuzzy_logic_engine.py</code>
Walidacja przedziałów	symulacja Monte Carlo dla weryfikacji obliczonych przedziałów ufności

Innowacyjność

Wkład pracy obejmuje trzy aspekty. Po pierwsze, jedno z nielicznych zastosowań logiki rozmytej typu drugiego w kontekście systemów rekomendacji w polskim środowisku akademickim – większość prac dotyczących IT2-FLS dotyczy sterowania, robotyki i finansów. Po

drugie, empiryczna weryfikacja pytania, czy modelowanie niepewności faktycznie poprawia jakość rekomendacji – z trzema systemami zamiast dwóch, co daje pełniejszy obraz: stała wiedza ekspercka kontra niepewność modelowana kontra wiedza uczona. Po trzecie, propozycja nowej metryki – szerokości przedziału ufności rekomendacji – jako miary pewności systemu; otwiera to drogę do rankingu świadomego pewności.

Uwaga – ANFIS jako rozszerzenie, nie zamiennik

W poprzedniej wersji listy tematów ANFIS figurował jako temat alternatywny (P4-ALT). Zgodnie z sugestią promotora („można do badania włączyć ANFIS i porównać te trzy systemy”) ANFIS zostaje włączony do tematu P4 jako trzeci porównywany system – nie zastępuje IT2-FLS, lecz uzupełnia badanie. Trzon pracy pozostaje przy wnioskowaniu przedziałowym i bibliotece `pyit2fls`.

Hipotezy badawcze

- **H1.** System typu drugiego osiąga wyższą metrykę Precision@K niż system typu pierwszego dla użytkowników z małą liczbą ocen – w warunkach rzadkich danych modelowanie niepewności ma większe znaczenie.
- **H2.** Szerokość przedziału ufności obliczonego przez system typu drugiego koreluje istotnie z błędem predykcji – szeroki przedział wskazuje rzeczywistą niepewność systemu.
- **H3.** ANFIS, ucząc parametry z danych, dorównuje lub przewyższa system typu pierwszego o stałych parametrach, a jego jakość rośnie wraz z liczbą próbek treningowych, osiągając plateau.
- **H4.** Koszt obliczeniowy systemu typu drugiego (zwłaszcza defuzyfikacji Karnika-Mendela) jest istotnie wyższy niż systemu typu pierwszego, lecz opłacalny tylko w wybranych scenariuszach (rzadkie dane, nowi użytkownicy).

Plan eksperymentu

Wykorzystanie istniejącego modułu typu pierwszego jako bazy odniesienia. Implementacja wariantu IT2-FLS z obszarem niepewności wyznaczonym jako odchylenie procentowe od funkcji typu pierwszego (parametr regulowany). Implementacja i trening ANFIS na badawczym zbiorze danych. Test na trzech rozłącznych podzbiorach użytkowników: o niskiej, średniej i wysokiej liczbie ocen. Porównanie wszystkich trzech systemów oraz wariantu hybrydowego (IT2 dla użytkowników o ubogiej historii, T1 dla pozostałych). Walidacja przedziałów ufności metodą Monte Carlo. Pomiar opóźnienia obliczeń dla każdego z systemów. Analiza statystyczna – test Friedmana z testem post-hoc.

Literatura

- Mendel, J. M. (2017). *Uncertain Rule-Based Fuzzy Systems: Introduction and New Directions*, wyd. 2. Springer.

- Karnik, N. N., Mendel, J. M. (2001). Centroid of a type-2 fuzzy set. *Information Sciences*, 132(1-4), 195–220.
- Wu, D. (2013). Approaches for reducing the computational cost of interval type-2 fuzzy logic systems. *IEEE Transactions on Fuzzy Systems*, 21(1), 80–99.
- Haghrah, A. A., Ghaemi, S. (2023). PyIT2FLS: A New Python Toolkit for Interval Type 2 Fuzzy Logic Systems. *arXiv:1909.10051*.
- Jang, J.-S. R. (1993). ANFIS: Adaptive-Network-Based Fuzzy Inference System. *IEEE Transactions on Systems, Man, and Cybernetics*, 23(3), 665–685.
- Chen, C., John, R. i in. Type-1 and Interval Type-2 ANFIS: A Comparison. *Materiały konferencyjne IEEE*.
- Castillo, O., Melin, P. (2008). *Type-2 Fuzzy Logic: Theory and Applications*. Springer.

7 Wiarygodność statystyczna – minimalna liczba użytkowników

Promotor wskazał, że wiarygodność (istotność) statystyczna testów jest dyktowana minimalnymi wymaganiami stosowanych testów oraz że „zwykle wystarcza kilkadziesiąt lub nawet kilkanaście użytkowników”. Poniżej przegląd literatury potwierdzający i precyzujący to stwierdzenie. Należy rozróżnić dwa rodzaje liczebności występujące w pracy: liczbę *użytkowników w badawczym zbiorze danych* (ewaluacja offline – metryki rankingowe) oraz liczbę *uczestników badania z udziałem ludzi* (ewentualna ocena ekspercka, badanie z użytkownikami).

Badania z udziałem użytkowników – liczby z literatury

- **Nielsen Norman Group** (przegląd zaleceń dotyczących liczebności prób w badaniach ilościowych) wskazuje, że **40 uczestników** jest liczbą odpowiednią dla większości badań ilościowych – przy poziomie ufności 95% i marginesie błędu 15%. Liczbę można obniżyć do **30** przy poziomie ufności 90% lub do **20–25** w projektach o ograniczonym budżecie, choć autorzy określają to jako podejście ryzykowne. Wniosek operacyjny tego samego źródła: *aby uzyskać statystycznie istotne wyniki, należy przebadać co najmniej 20 użytkowników; ciaśniejsze przedziały ufności wymagają większej liczby.*
Źródło: <https://www.nngroup.com/articles/summary-quant-sample-sizes/>
- **Knijnenburg & Willemsen**, *Evaluating Recommender Systems with User Experiments* – praca metodyczna wprost poświęcona eksperymentom z użytkownikami w systemach rekomendacji. Autorzy zwracają uwagę, że jedno z pierwszych badań user-centric systemu rekomendacji objęło zaledwie **19 uczestników** i mimo wykrycia istotnych efektów było *niedostatecznie zasilone (underpowered)* – wniosków nie dało się uogólnić. Kluczowa wskazówka metodyczna: liczba uczestników potrzebna dla danej mocy statystycznej rośnie *liniowo z liczbą warunków eksperymentu*, dlatego należy utrzymywać liczbę porównywanych warunków na niskim poziomie.
Źródło: <https://www.usabart.nl/portfolio/KnijnenburgWillemsen-UserExperiments.pdf>
- **Nielsen Norman Group**, *How Many Test Users in a Usability Study* – klasyczne rozróżnienie: badania *jakościowe* (wykrywanie problemów) wymagają około 5 użytkowników, natomiast badania *ilościowe* nastawione na statystykę wymagają co najmniej 20 użytkowników.
Źródło: <https://www.nngroup.com/articles/how-many-test-users/>
- Przegląd metodyczny w czasopismach medycznych (*Sample size: how many participants do I need in my research?*) podkreśla, że dokładna liczebność wynika z planowanej *mocy statystycznej* (zwykle 80%), zakładanej wielkości efektu i poziomu istotności ($\alpha = 0,05$) – liczbę należy wyznaczyć analizą mocy, a nie przyjmować arbitralnie.
Źródło: <https://pmc.ncbi.nlm.nih.gov/articles/PMC4148275/>

Wnioski dla obu prac

Ewaluacja offline (metryki rankingowe, testy Friedmana po użytkownikach).

Badawczy zbiór danych powinien obejmować rzędu **150–300 użytkowników** z odpowiednio bogatą historią – jest to liczba swobodnie osiągalna przez parametryzowany seeder, a jednocześnie wystarczająca, by test Friedmana po użytkownikach miał wysoką moc. Tu liczebność nie jest ograniczeniem – ogranicza ją wyłącznie realizm danych, nie ich pozyskanie.

Badanie z udziałem ludzi / ocena ekspercka (jeśli zostanie przeprowadzone).

Minimum **20 uczestników** dla uzyskania istotności statystycznej; **30–40 uczestników** jako wartość bezpieczna (poziom ufności 90–95%). Liczbę porównywanych warunków (modeli, systemów) należy utrzymać nisko – stąd rekomendowany zestaw 4–6 modeli w temacie D2 i 3–4 systemy w temacie P4.

Zasada nadrzędna. Ostateczną liczebność wyznacza analiza mocy statystycznej dla konkretnie wybranego testu (Friedman, Wilcoxon, ANOVA), przy założonej mocy 80% i $\alpha = 0,05$. Zostanie to opisane w rozdziale metodycznym każdej z prac.

8 Podział prac i harmonogram ramowy

Zakres	Realizacja
Wspólnie	Badawczy seeder seed_research, polecenie prepare_research_db, wspólny moduł metryk, model ExperimentRun, wspólny rozdział zestawiający wyniki obu prac.
Dawid Olko (D2)	Moduł rekomendacji LLM (prompting, embeddings, RAG), dobór i konfiguracja modeli, eksperymenty porównawcze LLM kontra metody klasyczne, ocena halucynacji i kosztu, badanie modeli polskich.
Piotr Smoła (P4)	Moduły IT2-FLS i ANFIS, przepisanie bazy reguł rozmytych, panel diagnostyczny FOU, eksperymenty porównawcze trzech systemów rozmytych, walidacja przedziałów Monte Carlo.

Każda praca ma własny tytuł i własne rozdziały. Wstęp obu prac może zawierać wspólny akapit o tym samym repozytorium i tym samym badawczym zbiorze danych. Ramowy harmonogram: (1) fundament wspólny; (2) implementacja modułów własnych; (3) eksperymenty i zbieranie wyników; (4) analiza statystyczna i redakcja; (5) wspólny rozdział zestawiający wyniki.

Podsumowanie

Tematy D2 i P4 zostały zatwierdzone. Niniejszy dokument uszczegóławia ich zakres w oparciu o faktyczną analizę aplikacji *SmartRecommender* oraz uwzględnia wszystkie cztery wskazówki promotora: język pracy (polski) i promptów (angielski), polskie modele Bielik i PLLuM jako punkt odniesienia w temacie D2, włączenie ANFIS jako trzeciego systemu w temacie P4 oraz oparcie liczebności prób na przeglądzie literatury. Oba tematy współdzielą fundament badawczy, co umożliwi bezpośrednie zestawienie wyników i – w razie interesujących rezultatów – wspólną publikację.